

Shopaholic – Price Tracker

ID echipa: F04

Pielcaru Riana-Maria

Iliescu Raluca Sandra

Documentație Tehnică

1. Rezumatul Proiectului

Proiectul constă în dezvoltarea unei aplicații desktop native, realizată în limbajul **Python**, cu scopul de a automatiza monitorizarea prețurilor pentru produse de pe platforma e-commerce *About You*. Sistemul extrage date în timp real, le stochează într-o bază de date locală (CSV) și efectuează o analiză comparativă pentru a detecta valorile minime din istoriul creat.

2. Configurarea Mediului

Pentru execuția și dezvoltarea soluției, au fost parcurse următoarele etape de configurare:

2.1. Instalarea Python

S-a utilizat versiunea **Python 3.13**. În faza de instalare, activarea opțiunii "**Add Python to PATH**" a fost obligatorie pentru a permite accesul la managerul de pachete pip direct din interfața de linie de comandă (CLI).

2.2. Instalarea Bibliotecilor Externe

Execuția proiectului depinde de următoarele module populare, instalate via Terminal:

- requests: Gestionarea cererilor HTTP către serverele externe.
 - beautifulsoup4: Analiza sintactică (parsing) a documentelor HTML.
 - pandas: Structurarea și manipularea datelor sub formă de tabele (DataFrames).
 - tkinter: Biblioteca standard pentru crearea interfețelor grafice (GUI) native.
-

3. Arhitectura Software

Codul sursă utilizează **Programarea Orientată pe Obiecte (POO)**, fiind structurat în două clase principale pentru a asigura modularitatea și separarea responsabilităților (Separation of Concerns).

Etapa I: Extragerea Datelor (Clasa ShopaholicScraper)

1. **Simularea Clientului:** Se definește un dicționar de tip headers (User-Agent) pentru a evita blocarea cererilor de către sistemele anti-bot ale serverului.
2. **Cererile HTTP:** Se utilizează requests.get() pentru a prelua codul sursă al paginii.
3. **Analiza DOM:** BeautifulSoup scanează structura HTML. Se utilizează selectori specifici identificați prin inspectarea browserului:
 - Titlul produsului: Eticheta <h1>.
 - Prețul: Eticheta cu atributul data-testid="finalPrice".
4. **Curățarea Datelor (Regex):** Se utilizează modulul re (Regular Expressions) pentru a corecta erorile de formatare (ex. inserarea spațiilor între cuvintele lipite prin pattern-ul ([a-z])([A-Z])).

Etapa II: Persistența Datelor (Pandas & CSV)

1. **Stocarea:** Datele extrase sunt organizate într-un dicționar și ulterior convertite într-un obiect de tip pandas.DataFrame.
2. **Gestiunea Fișierelor:** Se verifică existența fișierului istoric _preturi.csv.
 - Dacă fișierul nu există: Se creează unul nou, incluzând antetul (header).
 - Dacă fișierul există: Se utilizează modul append (mode='a') pentru a adăuga datele noi fără a suprascrie istoricul existent.

Etapa III: Logica de Monitorizare și Analiză

1. **Filtrarea:** La fiecare interogare, sistemul încarcă baza de date CSV și filtrează intrările specifice numelui produsului curent.
2. **Calculul Matematic:** Se utilizează funcția .min() asupra coloanei pret pentru a identifica cea mai mică valoare înregistrată în ultimele 30 de zile (sau de la începutul monitorizării).
3. **Evaluarea Condițională:** Se compară prețul live cu minimul istoric. Dacă valoarea actuală este strict mai mică, sistemul declanșează o notificare vizuală de tip alertă.

Etapa IV: Interfața Grafică (Clasa ShopaholicApp)

1. **Componente UI:** S-au implementat elemente de tip Label (pentru instrucțiuni și afișare preț), Entry (pentru introducerea URL-ului) și Button (pentru declanșarea fluxului de lucru).
2. **Loop-ul Principal:** Metoda mainloop() menține fereastra activă și gestionează evenimentele (click-urile utilizatorului).

4. Instrucțiuni de Rulare

Pentru a verifica integritatea și funcționalitatea proiectului, se vor urma pașii:

1. Se deschide folderul proiectului în **PyCharm** sau **VS Code**.
2. Se rulează scriptul principal:

Bash

python main.py

3. În fereastra deschisă, se introduce un link valid de produs (de pe About You).
4. Se acționează butonul **"Verifică"**. Rezultatele vor apărea instantaneu pe ecran și vor fi salvate automat în fișierul `istoric_preturi.csv`.

5. Referințe Bibliografice

<https://www.youtube.com/watch?v=qukjS96clB8>

<https://oxylabs.io/blog/how-to-build-a-price-tracker>

<https://scrapfly.io/blog/posts/how-to-build-a-price-tracker-using-python-web-scraping>

<https://www.youtube.com/watch?v=dmZ5SBIMxIE>

<https://www.geeksforgeeks.org/python/python-requests-tutorial/>

https://www.w3schools.com/python/module_requests.asp

<https://www.geeksforgeeks.org/python/implementing-web-scraping-python-beautiful-soup/>

<https://www.youtube.com/watch?v=bargNl2WeN4>

<https://www.youtube.com/watch?v=8dTpNajxaH0>

Codul sursă:

```
import requests # Gestionare cereri HTTP
from bs4 import BeautifulSoup # Analiza si extragere date din documente HTML
from datetime import datetime # Manipulare date si ore sistem
import pandas as pd # Analiza de date si manipulare tabele CSV
import os # Interactiune cu sistemul de fisiere local
import tkinter as tk # Creare interfata grafica utilizator (GUI)
from tkinter import messagebox # Afisare ferestre de dialog standard
import re # Procesare text prin expresii regulate

# Clasa pentru extragerea datelor de pe web
class ShopaholicScraper:
    def __init__(self):

        # Initalizare header pentru a simula un browser real
        self.headers = {
            "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/91.0.4472.124 Safari/537.36"
        }

    # Preluare nume articol si pret din url-ul specificat
    def extrage_date(self, url):
        try:

            # Cerere de tip http get catre site
            pagina = requests.get(url, headers=self.headers, timeout=10)
            if pagina.status_code == 200:
                soup = BeautifulSoup(pagina.content, 'html.parser')

                # Extragere nume din H1
                h1_element = soup.find("h1")
                if not h1_element:
                    return None

                # Extragere text cu spatiu intre elementele interne (curatare)
                nume_raw = h1_element.get_text(separator=" ", strip=True)

                # Corectie text lipit prin Regex (numele brand-ului era extras lipit de numele articolului)
                nume_corectat = re.sub(r'([a-z])([A-Z])', r'\1 \2', nume_raw)
                nume_final = " ".join(nume_corectat.split())

                # Localizare pret dupa atributul data-testid
                pret_element = soup.find("span", {"data-testid": "finalPrice"})
                if not pret_element:
                    return None

                # Conversie text pret in valoare numerica float
                pret_raw = pret_element.get_text()
```

```

        # Curatare de caractere non-numerice
        pret_numeric = float(pret_raw.replace('lei', '').replace(',', '.').replace('\xa0', '').strip())

        return {
            "nume": nume_final,
            "pret": pret_numeric,
            "data": datetime.now().strftime("%Y-%m-%d %H:%M")
        }
    except Exception as e:
        print(f"Eroare scraping: {e}")
        return None

# Clasa pentru a gestiona interfata si logica aplicatiei
class ShopaholicApp:
    def __init__(self, root):

        # Configurare fereastră principală și motor scraping
        self.root = root
        self.root.title("Shopaholic - Price Tracker")
        self.root.geometry("600x450")
        self.scrapper = ShopaholicScraper()

        # Interfața grafică (GUI): etichete, câmp intrare și buton
        tk.Label(root, text="SHOPAHOLIC", font=("Arial", 18, "bold")).pack(pady=10)

        tk.Label(root, text="Introdu link-ul produsului de pe About You:", font=("Arial", 10)).pack(pady=5)
        self.url_entry = tk.Entry(root, width=70)
        self.url_entry.pack(pady=10)

        # Buton pentru verificare
        tk.Button(root, text="Verifică", command=self.proceseaza_link,
                  bg="black", fg="white", font=("Arial", 10, "bold")).pack(pady=15)

        # Zona afisare rezultate
        self.lbl_info = tk.Label(root, text="", font=("Arial", 10), justify="center")
        self.lbl_info.pack(pady=20)

        # Executie flux: citire url, scraping, comparare pret si salvare
        def proceseaza_link(self):
            url = self.url_entry.get()
            if not url:
                messagebox.showwarning("Atenție", "Te rugăm să introduci un link!")
                return

            date_noi = self.scrapper.extrage_date(url)

            if date_noi:
                nume_fisier = "istoric_preturi.csv"
                msg_status = f"Produs: {date_noi['nume']}\nPreț actual: {date_noi['pret']} RON"

```

```

# Logica de comparare a datelor din csv
if os.path.isfile(ume_fisier):
    df = pd.read_csv(ume_fisier)
    istoric_produs = df[df['nume'] == date_noi['nume']]

    if not istoric_produs.empty:
        minim_vechi = istoric_produs['pret'].min()

        # Verificare daca pretul actual este noul minim
        if date_noi['pret'] < minim_vechi:
            msg_status += f"\n\n NOU MINIM DETECTAT! (Anterior: {minim_vechi} RON)"
        else:
            msg_status += f"\n\nMinimul înregistrat în istoric: {minim_vechi} RON"
        else:
            msg_status += "\n\nAcesta este primul preț înregistrat pentru acest produs."

# Salvarea datelor prin adaugare la finalul fisierului (mode append)
df_nou = pd.DataFrame([date_noi])
df_nou.to_csv(ume_fisier, mode='a', index=False, header=not os.path.isfile(ume_fisier))

self.lbl_info.config(text=msg_status, fg="blue")
else:
    messagebox.showerror("Eroare", "Nu am putut extrage datele. Verificați link-ul!")

# Pornire aplicatie
if __name__ == "__main__":
    root = tk.Tk()
    app = ShopaholicApp(root)
    root.mainloop()

```

